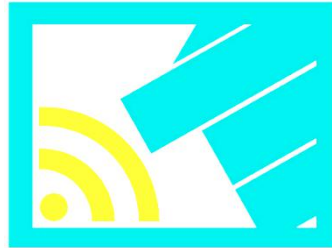


優方科技股份有限公司 Best Solution Inc.



優方软件包架構說明 2022/04/11 V512

目錄

1.0 軟件包架構說明.....	3
1.1 主程序 MAIN_PROGRAM 介紹.....	3
1.2 外部函数介紹.....	3
1.3 外部函數工作說明.....	3
2.0 軟件包流程圖.....	4
2.1 工作流程圖.....	4
2.2 工作流程圖說明.....	5
2.2.1 初始化流程說明.....	5
2.2.2 工作流程說明.....	5
2.3 休眠流程圖.....	6
2.4 休眠流程圖說明.....	7
2.4.1 休眠標誌位.....	7
2.4.2 休眠函數說明.....	7
2.4.3 休眠流程說明.....	7
3.0 軟件包客製程序範例.....	8
3.1 創建 C/ASM 程序檔案.....	8
3.2 編寫 EXT 檔案.....	9
3.3 配置 INC 檔案.....	10
4.0 軟件包結構圖.....	11
4.1 優方軟件包資料夾結構.....	11
4.2 V512 軟件包引用關係圖.....	13
4.4 新版軟件包優點.....	15

1.0 軟件包架構說明

1.1 主程序 MAIN_PROGRAM 介紹

MAIN_PROGRAM 為軟件包的核心控制程序，可以實現 IC 硬件初始化，執行各功能的外部函數以及對 IC 工作與休眠的流程控制。用戶可將自定義的功能函數（如觸摸功能、AD 採集功能、串口通信等等功能）加入到 MAIN_PROGRAM，主程序會通過軟件的流程控制依次實現各個功能。

1.2 外部函数介紹

V512 軟件包共有 5 個外部功能函數，每個外部功能函數執行的功能都不相同。用戶可根據實際需要將自定義函數的加入以下五個外部功能函數之中：

1.EXTEND_FUNCTION_INITIAL()

執行外部初始化函數，只運行一次。

2.EXTEND_FUNCTION()

執行外部工作函數，若沒開啟省電模式，程序會在此循環執行。

3.EXTEND_FUNCTION_HALT_PREPARE()

準備進入待機前的執行的函數。

4.EXTEND_FUNCTION_HALT_WAKEUP()

待機模式被中斷、IO 喚醒之後執行的函數。

5.EXTEND_FUNCTION_RETURN_MAIN()

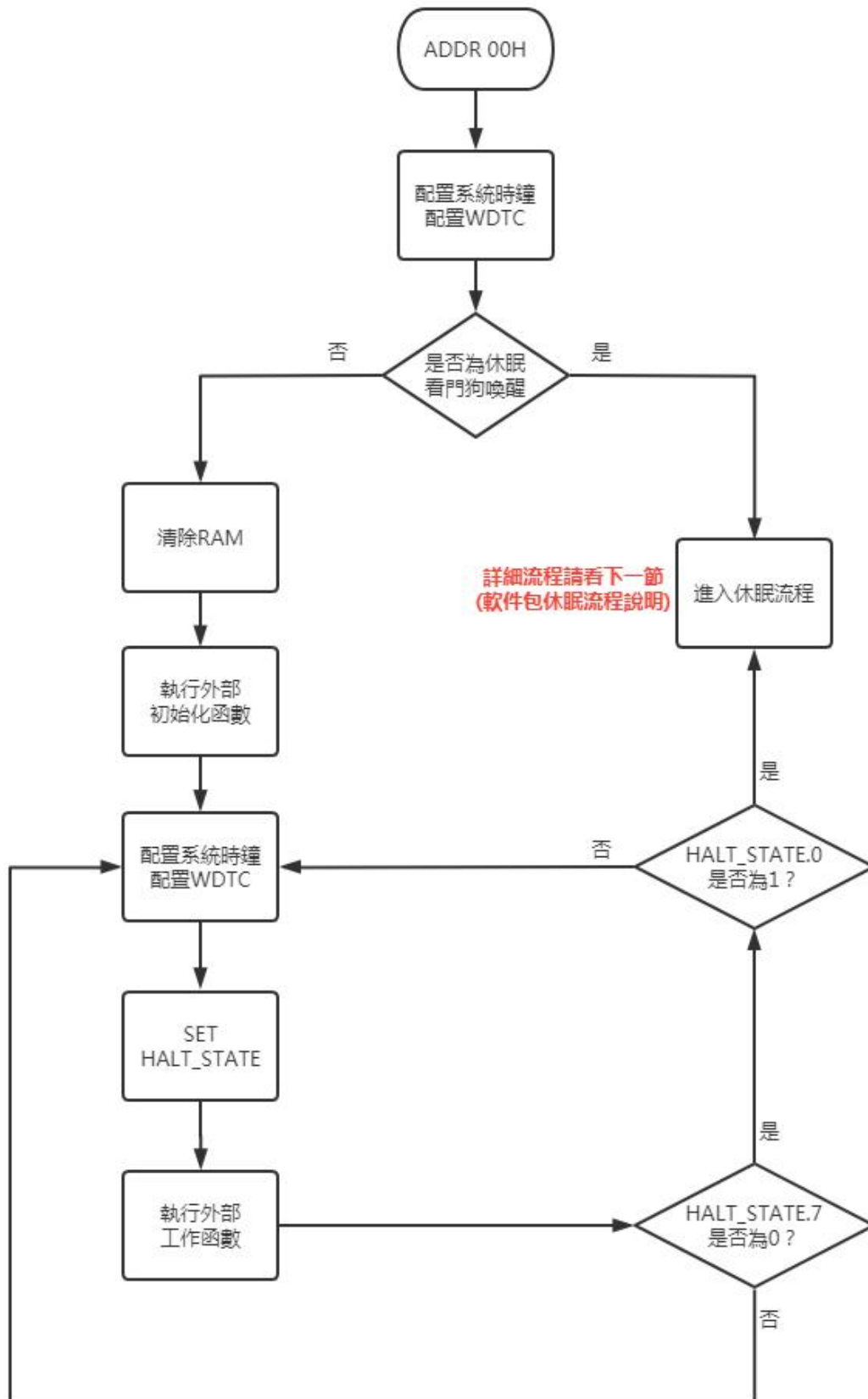
待機模式切回工作模式時執行的函數。

1.3 外部函數工作說明

當 MAIN_PROGRAM 在執行某外部函數時，軟件包會從 **EXTEND_FUNCTION_1A_XXXX** 到 **EXTEND_FUNCTION_2F_XXXX** 依次向下執行，其中的 1A~2F 表示該外部函数区不同的入口。用戶可在 MAIN_PROGRAM.INC 中為自定義的功能函数設置不同的函數入口，當軟件包執行該外部功能函數時，會根據 1A~2F 的順序依次執行。（V512 軟件包的觸摸包也是屬於自定義功能函數，可以不添加觸摸功能）

2.0 軟件包流程圖

2.1 工作流程圖



2.2 工作流程圖說明

2.2.1 初始化流程說明

IC 上電或者復位的時候軟件包會從 00H 往下執行，在配置系統時鐘和看門狗溢出時間後程序會根據 PDF 標誌位與 T0 標誌位來判斷，當這兩個標誌位均為 1 時表示 IC 在休眠被看門狗溢出喚醒。

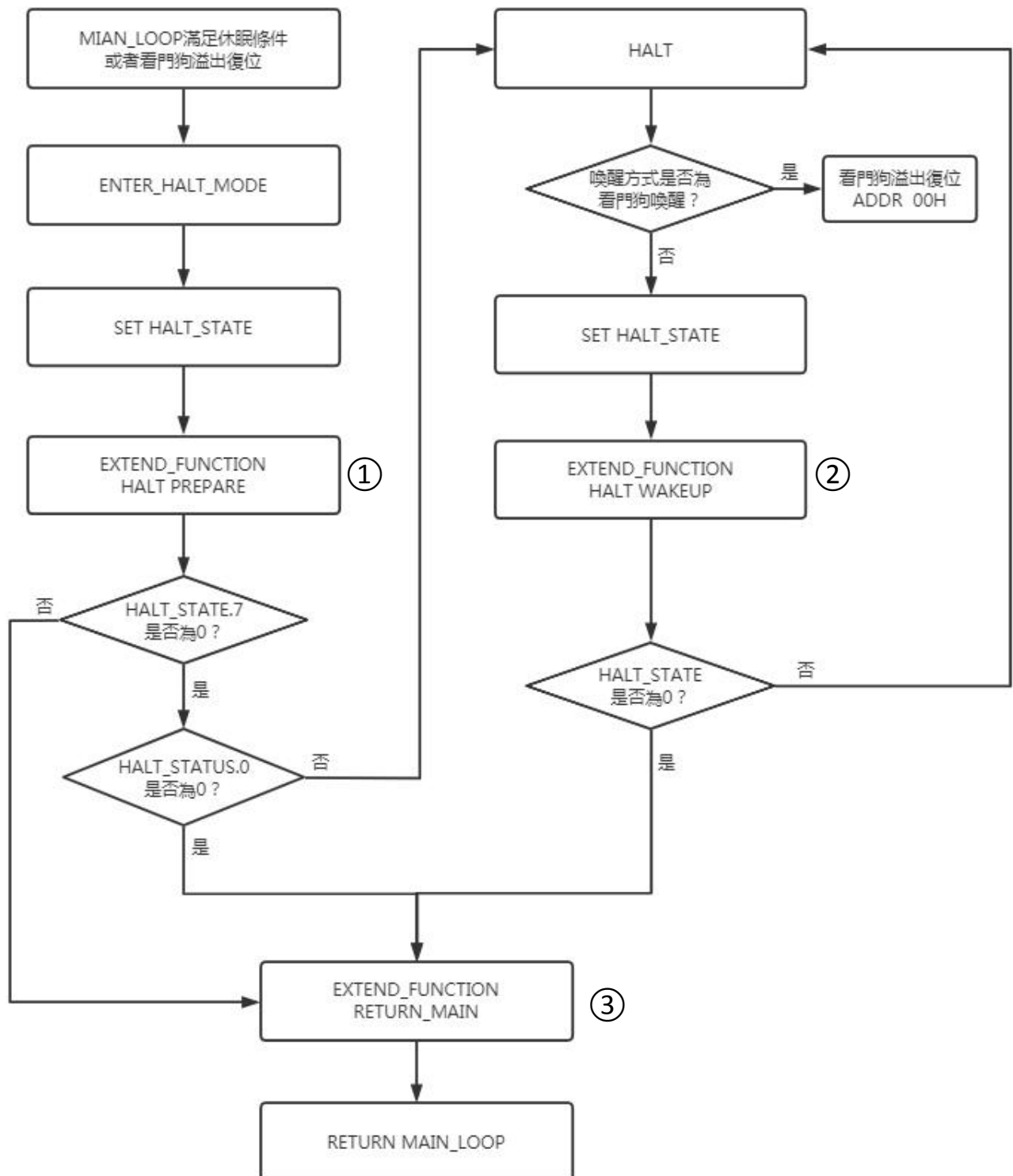
如果軟件包是休眠情況下被看門狗喚醒程序會直接走右邊的路線進入休眠流程，上電或者其他情況導致的復位則會走左邊的初始化流程，清 RAM 後重新配置時鐘和看門狗溢出時間，之後執行所有的初始化函數。

2.2.2 工作流程說明

當軟件包初始化完成後，軟件包會進入工作流程 MAIN_LOOP 中。若軟件包未開啟省電模式，軟件包會循環執行 MAIN_LOOP 中的工作流程。在配置系統時鐘和看門狗溢出時間之後將休眠標誌位 HALT_STATE 置一，執行用戶所有的外部工作函數: **EXTEND_FUNCTION**。

執行完外部工作函數後，軟件包會根據休眠標誌位 HALT_STATE 判斷程序是否滿足休眠條件。若開啟休眠模式的話觸摸包底層會對 HALT_STATE 操作，讓軟件包滿足休眠條件後可以進入休眠。若未開啟休眠模式用戶需要自己在外部工作函數對 HALT_STATE 操作才能進入休眠，若用戶無操作軟件包將會循環執行工作流程。

2.3 休眠流程圖



2.4 休眠流程圖說明

2.4.1 休眠標誌位

V512 軟件包若開啟省電模式，程序會根據休眠標誌位 **HALT_STATE** 控制軟件包的休眠流程。以下為休眠標誌位最高位與最低位的解釋說明：

HALT_STATE.bit0（可讀寫）

該位為程序能否休眠判斷位。若改位為 1 表示程序滿足休眠條件，之後會進入休眠。若該標誌位為 0 則表示程序不滿足休眠條件，之後程序會返回工作流程。

HALT_STATE.bit7（可讀寫）

該位為休眠標誌位 RAM 操作有效位。若該標誌位為 0 則視為休眠標誌位有效，軟件包會根據 **HALT_STATE.0** 決定軟件包能否進入休眠。若該標誌位為 1 則操作無效，軟件包將不會根據休眠判斷位進行流程控制。因此在修改休眠標誌位時必須將 **HALT_STATE.7** 清零。

2.4.2 休眠函數說明

軟件包進入休眠流程後，會根據流程圖執行三個外部休眠函數。每個外部休眠函數的執行方式如下所示：

① **EXTEND_FUNCTION_HALT_PREPARE()**

外部休眠準備函數：IC 進入休眠流程後執行

② **EXTEND_FUNCTION_HALT_PREPARE()**

外部休眠喚醒函數：IC 從休眠模式被中斷、IO 喚醒之後執行

③ **EXTEND_FUNCTION_RETURN_MAIN()**

外部休眠返回工作函數：IC 從休眠模式返回工作模式時執行

2.4.3 休眠流程說明

在執行外部休眠準備與喚醒函數前，軟件包會先執行 SET **HALT_STATE** 的操作。即將 **HALT_STATE.7** 與 **HALT_STATE.0** 置一，用戶可在自定義的外部休眠函數中對 **HALT_STATE** 標誌位進行操作。軟件包會根據用戶執行外部函數後的 **HALT_STATE** 來對程序進行休眠流程控制。

若用戶在**外部休眠準備函數**中沒有對 **HALT_STATE** 操作，軟件包將無法休眠。而是直接返回外部工作函數 **MAIN_LOOP**，若用戶在**外部休眠喚醒函數**中沒有對 **HALT_STATE** 操作，IC 在執行完所有外部休眠喚醒函數後將繼續進入休眠。

3.0 軟件包客製程序範例

以在優方的軟件包中添加一個 AD 採集客製程序為例，用戶可根據以下的步驟創建 C/ASM、EXT、INC 檔案並添加到軟件包架構當中，從而實現製作一個帶 AD 採集的軟件包。

3.1 創建 C/ASM 程序檔案

新建一個 C/ASM 文檔並在其中編寫 AD 採集程序，並將初始化函數與工作函數分開編寫，軟件包上電執行完所有的初始化函數，會循環執行所有的工作函數，C 語言的範例代碼如下：

```
#include "AD_FUNCTION.INC"
//=====
//*****
//=====

unsigned char  AD_VALUE_L = 0;
unsigned char  AD_VALUE_H = 0;

void AD_FUNCTION_INITIAL()
{
    // A/D//
    _sadc0 = 0b00010000; //設置輸入通道為AN0
    _sadc1 = 0b00001010; //外部通道,UDD,fsys/4
    _ace0 = 1;           //设置PC0为AN0通道
    _adcen = 1;
}

//=====
//*****
//=====

void AD_FUNCTION_WORK()
{
    _start = 0;
    _start = 1;
    _start = 0;
    while(!_adbz);
    AD_VALUE_L = _sado1;
    AD_VALUE_H = _sadoh;
}
```


3.2 編寫 EXT 檔案

為了讓軟件包中功能文檔的函數與參數能夠被其他的檔案使用，可通過為每一個 C/ASM 檔案添加對應的 EXT 檔案。如 MIAN_PROGRAM.ASM、USER_PROGRAM.ASM (C)、BSXXXXXC_CTOUCH.OBJ 這些文檔都有一一對應的 EXT 檔案，這些 EXT 檔案中會存放自身函數或變量的外部聲明，只要在頭文件中包含這些 EXT 檔案就可以直接調用這些功能文檔內的函數與變量。

我們在添加客製功能的時候也需要添加一個 EXT 文檔，讓其他文檔能夠通過這個 EXT 文檔來調用我們添加的客製功能函數與參數，下面為 EXT 文檔聲明外部函數與變量的方式。

聲明彙編函數: EXTERN _XXX :near

聲明彙編參數: EXTERN _A :BYTE

聲明 C 語言函數: extern void XXX();

聲明 C 語言參數: extern unsigned char A;

為了讓 EXT 檔案能同時識別彙編與 C 語言的聲明，可以通過使用;;;/*與;;*/來劃分 C 語言與彙編程序區域，當編譯器為彙編時，/*與*/將會被注釋，從而能夠編譯注釋符之間的程序。反之將跳過/*與*/注釋符之間的彙編程序（一個程序/變量可同時進行彙編和 C 語言的外部聲明）。

將 AD 採集程序的初始化、工作函數以及儲存 AD 值的兩個全局變量 AD_VALUE 在 EXT 檔案中進行聲明，範例代碼如下圖紅框所示：（可以從別의 EXT 文檔複製並修改 XXX_DEF、函數名、參數名）

```
;;;;;/*
ifndef AD_FUNCTION_DEF
    EXTERN   _AD_FUNCTION_INITIAL       :NEAR
    EXTERN   _AD_FUNCTION_WORK        :NEAR

    EXTERN   _AD_VALUE_H               :BYTE
    EXTERN   _AD_VALUE_L               :BYTE
endif
;;;;;*/

;;;;;///* ----- */
;;;;;///* C exported function,register       */
;;;;;///* ----- */

;;;;;/*
ifdef _INCLUDE_C_
;;;;;*/
;;;;;//-----
#ifndef AD_FUNCTION_DEF
    extern void   AD_FUNCTION_INITIAL();
    extern void   AD_FUNCTION_WORK();

    extern unsigned char   AD_VALUE_H;
    extern unsigned char   AD_VALUE_L;
#endif
;;;;;//-----
;;;;;/*
endif
;;;;;*/
```

3.3 配置 INC 檔案

優方軟件包內所有的 EXT 檔案都會包含在 GLOBE_INCLUDE.INC 中，同時將對應的 MCU 硬件資源存在 GLOBE_VARIES.INC 中。因此軟件包內所有的 INC 文檔只要包含這兩個頭文件便可以包含所有的 EXT 文檔以及對應 MCU 硬件資源。

但是這樣會讓程序也包含自身的 EXT 檔案從而對調用自身函數的外部聲明出現報錯。因此需要在 EXT 檔案中使用預編譯 `ifndef/endif` 語句來規避這個問題。首先在對應的 INC 文檔中定義一個 `XXX_DEF` 作為標記，之後在對應的 EXT 檔案中使用預編譯判斷是否是自身的頭文件，如果是自身的頭文件就不進行外部聲明，從而規避掉這個錯誤。以下為配置步驟：

步驟一：在 AD_FUNCTION.INC 文檔中定義 `AD_FUNCTION_DEF`，並包含軟件包的兩個全局頭文件 `GLOBE_VARIES.INC` 和 `GLOBE_INCLUDE.INC`，實現全局變量的調用。

```
#define AD_FUNCTION_DEF
#include "..\GLOBE\GLOBE_VARIES.INC"
#include "..\GLOBE\GLOBE_INCLUDE.INC"
```

步驟二：在 AD_FUNCTION.EXT 文檔中使用 `ifndef/endif` 語句，避免由於對自身函數與變量的外部聲明早餐編譯錯誤。

```
;;;;/*
ifndef AD_FUNCTION_DEF
    EXTERN  _AD_FUNCTION_INITIAL    :NEAR
    EXTERN  _AD_FUNCTION_WORK      :NEAR

    EXTERN  _AD_VALUE_H            :BYTE
    EXTERN  _AD_VALUE_L            :BYTE
endif
;;;;*/

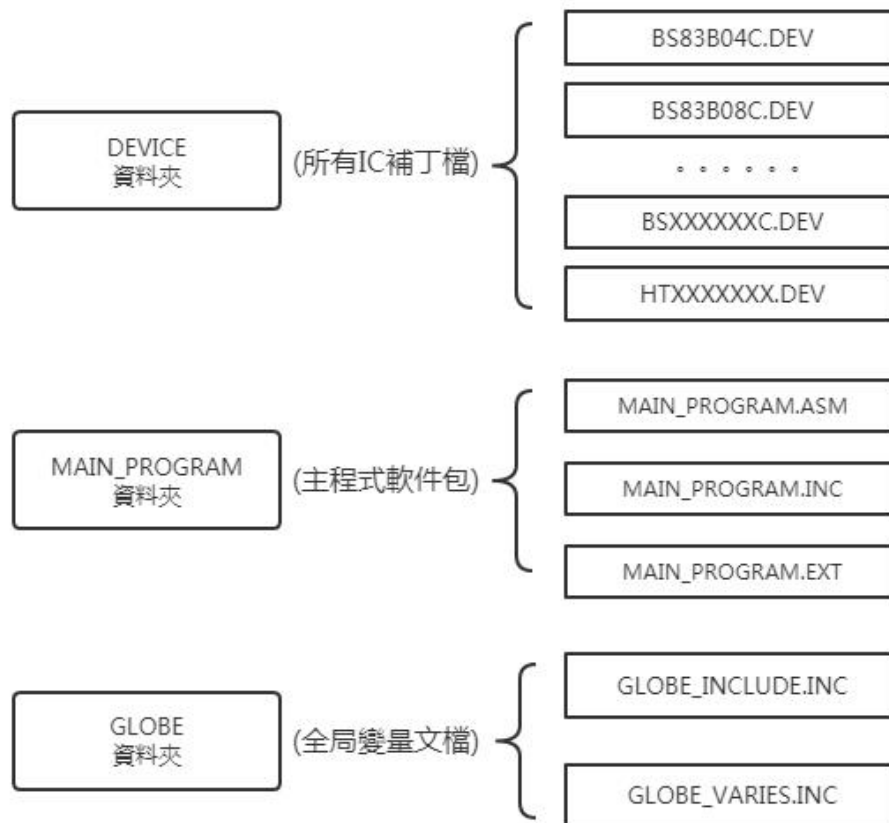
;;;;//----- */
;;;;//C exported function,register */
;;;;//----- */
;;;;/*
ifdef _INCLUDE_C_
;;;;*/
;;;;//-----
#ifndef AD_FUNCTION_DEF
    extern void  AD_FUNCTION_INITIAL();
    extern void  AD_FUNCTION_WORK();

    extern unsigned char  AD_VALUE_H;
    extern unsigned char  AD_VALUE_L;
#endif
;;;;//-----
;;;;/*
endif
;;;;*/
```

4.0 軟件包結構圖

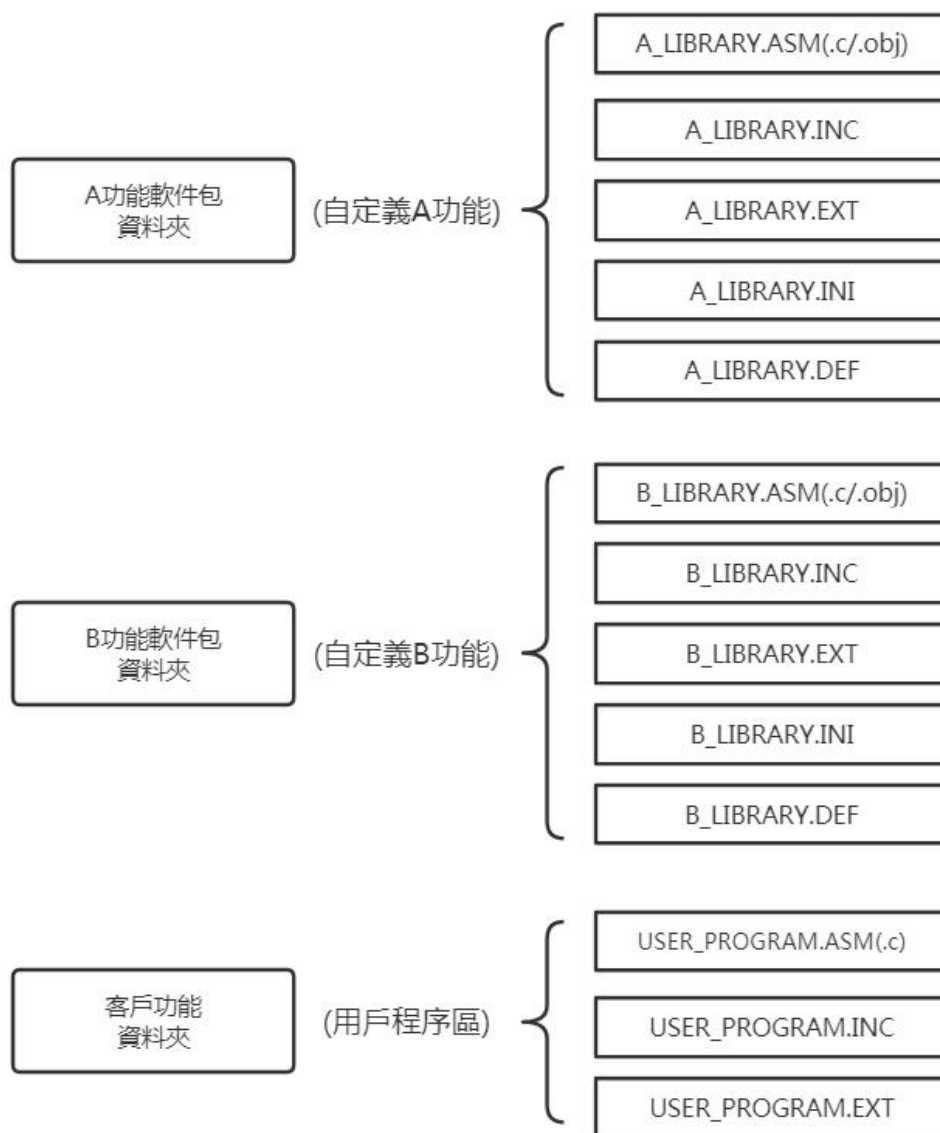
4.1 優方軟件包資料夾結構

V512 版本軟件包的資料夾分為兩個部分，第一部分是軟件包架構相關資料夾，用於存放軟件包正常工作的必要文件，第二部分是用戶自定義程序資料夾，用於編寫軟件包的客製功能。如下圖所示



圖一.軟件包架構相關資料夾

- ①DEVICE 資料夾:用於存放對應 IC 的 DEV 文檔，其中的 DEV 文檔會對 MCU 內部資源進行定義宣告的，因此軟件包在使用不同 IC 時只需要修改該資料夾的 DEV 檔案即可。
- ②MAIN_PROGRAM 資料夾:用於存放軟件包主程序，通過執行 MAIN_PROGRAM.ASM 中的程序實現軟件包的運作，用戶可通過 MAIN_PROGRAM.INC 將自定義的外部函數加入到主程序的循環中，EXT 檔案的則是對主程序的標誌位進行外部聲明。
- ③GLOBE 資料夾:用與存放軟件包全局變量文檔。第一個是 GLOBE_INCLUDE.INC，其包含了軟件包所有的 EXT 文檔，等於包含了軟件包所有的共用函數與變量的外部聲明。第二是 GLOBE_VARIES.INC 其包含 MCU 的硬件宣告 DEV 文檔，等於包含了軟件包對應 MCU 的硬件信息。



圖二.用戶自定義程序資料夾

- ①功能軟件包資料夾:用於存放用戶編寫的功能函數，例如觸摸包就屬於一種功能軟件包。由於觸摸包文檔為.OBJ形式，因此一般的方法無法修改其中的變量，需通過INI文檔來間接修改其中的參數(MAIN_PROGRAM.INC中也需要包含對應的INI文檔)。根據上文3.4章添加客製功能範例配置INC文檔和EXT文檔後便可將客製功能加入軟件包中。

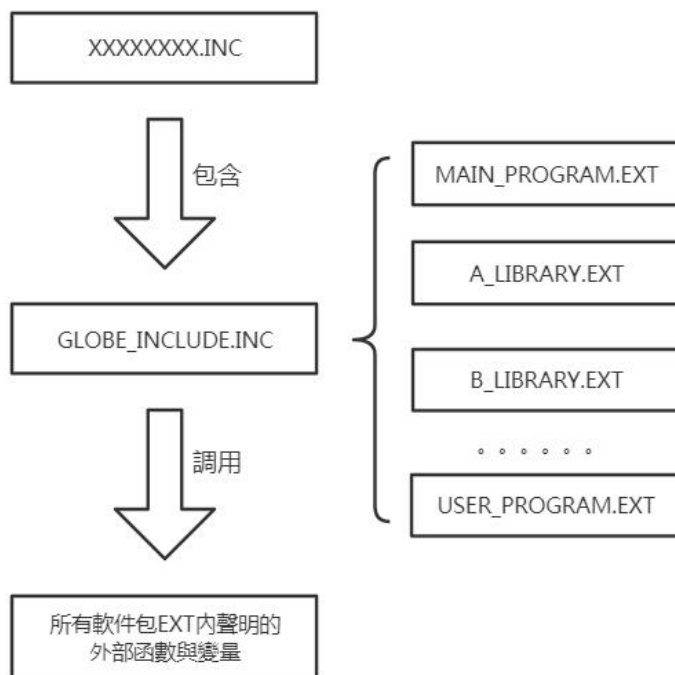
DEF文檔為程序擴充文檔，例如編寫IIC程序獲取0X10的數據，但是想要獲取0X12的地址時需要重新配置IIC的程序，如果將獲取0X10和獲取0X12的程序封裝為DEF文檔後，可根據實際情況外掛不同的DEF文檔。該文檔可根據需求選擇是否添加。

- ②客戶功能資料夾:用於存放客戶的程序代碼，用戶可在USER_PROGRAM.ASM(.C)中編寫自己的程序,同時可以直接調用其他文檔的函數與變量為自身使用。

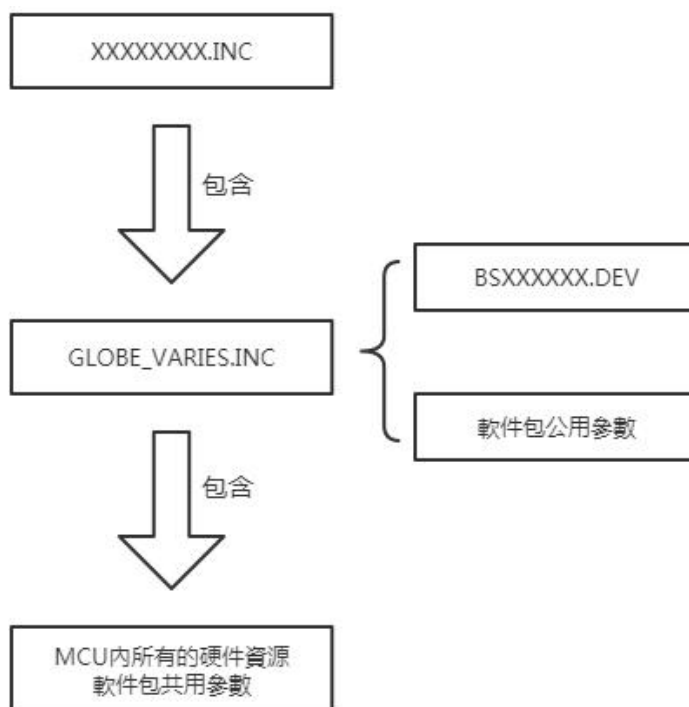
4.2 V512 軟件包引用關係圖

V512 軟件包的以 GLOBE 資料夾中的 GLOBE_INCLUDE.INC、GLOBE_VARIES.INC 為紐帶，通過包含這兩個頭文件實現對軟件包各個文檔的引用與連接。

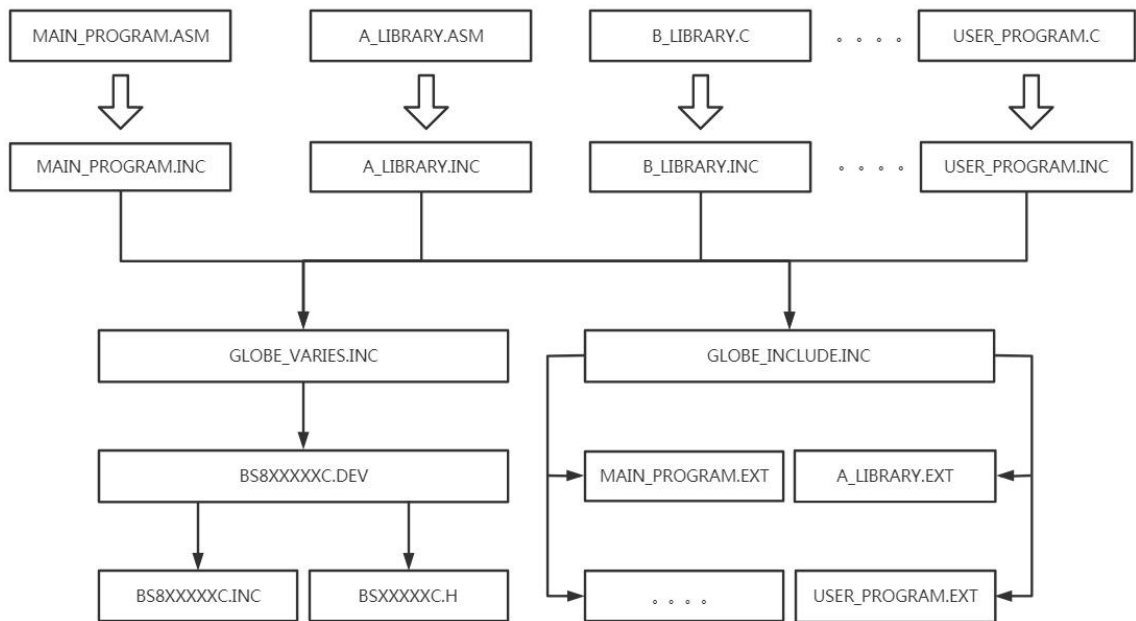
一. GLOBE_INCLUDE. INC 內包含了所有的 EXT 文檔，軟件包內的文檔只要包含了該頭文件後就可以間接調用軟件包內所有聲明過的外部函數與變量，具體的關係圖如下



二. GLOBE_VARIES. INC 內包 MCU 的資源 DEV 檔，軟件包內的頭文件只要包含了該頭文件後就可以包含軟件包內的對應 MCU 的硬件資源與共用參數，具體的關係圖如下



三. 使用 V512 軟件包在新增功能代碼時，只需根據 3.4 章教程配置 EXT 文檔與 INC 文檔並加入 MAIN_PROGRAM 之後，在自身 INC 文檔包含 GLOBE 資料夾內的這兩個紐帶頭文件即可完成功能移植，無需改動原有配置的頭文件。關係圖如下所示：



軟件包無論新增彙編或 C 程序，只需要將配置的 EXT 的頭文件放入 GLOBE_INCLUDE. INC 中，原有的程序便可以直接調用新加入的程序的功能或參數，從而讓移植新功能或程序更加便捷方便。

4.4 新版軟件包優點

優方軟件包具有完整的程序架構，還具備極強的可移植性。當用戶需要添加 C 語言或彙編的客製功能時，無需對原有的框架進行大幅整改，只需要配置 EXT 文檔以及修改關鍵的頭文件就能將新功能添加進軟件包中。

該版軟件包擺脫了觸摸包束縛，用戶可以不使用觸摸包功能函數，讓該軟件包不僅僅用在觸摸領域，也可以運用在其他低功耗軟件包領域。另外新版的軟件包新增了三個外部休眠函數，用戶可根據實際情況來編寫程序，極大的增強代碼的可操作空間。以下為該版軟件包新增的檔案說明：

① 新增檔案說明 (.DEV)

由於有不同 IC 內部寄存器的名字可能不相同，舊版本的軟件包採用將所有寄存器統一命名並存在 MAIN_PROGRAM.MCU 文檔內，但如果有新增 IC 的情況則需要將所有之前所有軟件包內的 MAIN_PROGRAM.MCU 更新。

新軟件包內的 DEV 文檔對應自身的 MCU，如 BS83B04C、BS83B08C、BS83B12C 軟件包都有獨立的 DEV 文檔與之一一對應，該文檔會將 MCU 的寄存器統一命名供軟件包使用。如果有新增 IC 只需新增一個 DEV 文檔，不需要對原來的軟件包進行修改。

② 新增檔案說明 (.DEF)

該文檔用於擴充軟件包的程序，尤其針對通訊專用代碼 (IIC/UART)。例如我們希望使用 IIC 獲取軟件包不同位置的地址，只需要編寫獲取特定地址 DEF 文檔。在使用時將該文檔掛到軟件包中，便可根據實際情況選擇掛任意 DEF 文檔獲取不同位置數據

③ 新增檔案說明 (.INI)

該文檔的作用是用來協助 OBJ 檔案內的數值修改，同時解決舊版本軟件包必須在 MAIN_PROGRAM 中加載觸摸包參數的問題。讓軟件包擺脫一定要使用觸摸包的情況。該文檔的好處在於：可將軟件包大多數功能移至 INI，讓客戶能夠在 INI 文檔中將不使用的功能關閉從而節省 ROM 空間，並讓客戶能夠修改部分底層的觸摸參數。